JobMatch AI

A Gemma-Powered Full-Stack RAG Job Recommendation System

Applied Generative AI (BUAN361)

Aadit Shah and Ananya Chenny Rahul

April 2026

Contents

1 Abstract **2**

2 Problem Statement **2**

3 Project Objectives **2**

4 Application Overview **2**

5 Core Features **3**

5.1 Resume-Aware Job Matching 3

5.2 Resume Parsing and Quality Gate 3

5.3 RAG Retrieval Debugging 4

5.4 Job Browsing and Data Administration 4

5.5 Application Workflow Tools 4

6 System Architecture **5**

7 Data and Ingestion Pipeline **5**

8 RAG and Matching Methodology **5**

8.1 Query Construction 5

8.2 Retrieval 6

8.3 Reranking 6

8.4 Gemma Generation 6

9 Implementation Details **6**

9.1 Backend Endpoints 6

9.2 Frontend 6

9.3 Security and Robustness 7

10 Evaluation Methodology **7**

11 Preliminary Evaluation Results **7**

12 Testing and Validation **8**

13 Limitations **9**

14 Future Scope **9**

15 Conclusion **9**

A Appendix: Screenshot Checklist **9**

B Appendix: Main Technologies **10**

1 Abstract

JobMatch AI is an end-to-end full-stack application that uses retrieval augmented generation (RAG) to help users discover relevant jobs, understand why those jobs fit their profile, and prepare stronger applications. The application combines a FastAPI backend, a browser-based frontend, SQLite persistence, Pinecone vector retrieval, external job ingestion, and Gemma-based generation. Users can upload a resume, receive personalized job matches, browse and save jobs, track application progress, generate cover letters, improve resumes, tailor resumes to specific job descriptions, and compose recruiter emails.

This report describes the problem, architecture, implementation, evaluation methodology, preliminary metrics, limitations, and future scope of the system. The focus is not only that generative AI was used, but that the app implements a working full-stack GenAI workflow with retrieval, ranking, stateful user features, API boundaries, safety checks, and measurable evaluation hooks.

2 Problem Statement

Job seekers face a search and personalization problem. Public job boards contain thousands of postings, but most search interfaces rely on keyword matching and leave the user to manually decide whether a role fits their skills, experience, location preference, salary expectations, and application goals. This is especially inefficient for early-career applicants who may not know which titles to search for or how to translate their resume into the language used in job descriptions.

The project aims to solve the following problem:

> How can we build a practical GenAI application that reads a candidate's resume, retrieves relevant job opportunities, explains the match quality, and supports the downstream application workflow?

The system must therefore do more than generate text. It must maintain a job corpus, retrieve suitable roles, rank them consistently, expose the results through a usable frontend, store user actions, and generate career documents using grounded job and resume context.

3 Project Objectives

The main objectives were:

- Build a full-stack GenAI application rather than a standalone prompt demo.
- Use Gemma as the application LLM for resume parsing, recommendation explanation, cover letter generation, resume feedback, and recruiter communication.
- Implement a RAG pipeline where job postings are normalized, indexed, retrieved, reranked, and passed to the model as grounded context.
- Provide user-facing workflows such as resume upload, job browsing, bookmarking, application tracking, and email support.
- Add evaluation infrastructure for retrieval quality, generation quality, diversity, latency, and reranking ablation.
- Include defensive handling for prompt injection and low-quality resume uploads.

4 Application Overview

JobMatch AI starts from the user's resume and optional constraints. The user signs in, uploads a PDF resume, reviews or edits extracted skills and constraints, and asks the system to find suitable jobs. The backend canonicalizes the user query, retrieves jobs from the indexed corpus, reranks candidates, and generates a structured recommendation response using Gemma. The frontend renders the response as job cards with match scores, metadata, explanation fields, and actions.

The broader application workflow includes:

1. User signs in with Google SSO.
2. User uploads a resume PDF.
3. Backend extracts profile details and validates that the PDF looks like a resume.
4. User optionally adds role, location, salary, work type, and skills constraints.
5. Backend retrieves and reranks job postings using the RAG pipeline.
6. Gemma produces a structured recommendation response.
7. User can bookmark roles, update application status, generate cover letters, tailor resumes, and email results.

Screenshot placeholder: `screenshots/landing.png`
Add the app screenshot with this filename before final PDF compilation.

Figure 1: Landing page and authentication entry point.

Screenshot placeholder: `screenshots/job-results.png`
Add the app screenshot with this filename before final PDF compilation.

Figure 2: Personalized job recommendation results.

5 Core Features

5.1 Resume-Aware Job Matching

The central feature is personalized job matching. The frontend collects a resume and optional constraints, while the backend builds a search query from the profile. The system extracts skills, desired roles, location, salary expectations, education, experience, work authorization, and free-form preferences where available. Retrieved jobs are then summarized into a user-facing recommendation.

5.2 Resume Parsing and Quality Gate

The backend uses PDF extraction to read resume text, then applies validation before sending cleaned text into Gemma. The validation checks whether the upload resembles a real resume and flags repeated prompt-control content. This prevents the system from treating arbitrary or malicious PDF text as a valid resume.

5.3 RAG Retrieval Debugging

The application exposes debug endpoints and a frontend debug modal. These show the canonical query, retrieved candidates, lexical overlap, prompt preview, model information, and retrieval fingerprint. This is useful because RAG behavior can otherwise be difficult to inspect from the final generated answer alone.

5.4 Job Browsing and Data Administration

Users can browse available jobs without uploading a resume. The browse view supports keyword, location, and industry filters. The admin panel can check job statistics, refresh configured external sources, and manually add a job. This turns the system into a data-backed application rather than a single-shot chat interface.

5.5 Application Workflow Tools

JobMatch AI includes user state and follow-through features:

- Bookmark jobs and remove bookmarks.
- Save jobs into an application tracker.
- Update statuses such as saved, applied, interviewing, offered, or rejected.
- Add notes for each tracked application.
- Generate cover letters for selected jobs.
- Tailor resumes and identify missing keywords for a job description.
- Compose recruiter emails with a tailored resume draft.

Screenshot placeholder: `screenshots/my-applications.png`
Add the app screenshot with this filename before final PDF compilation.

Figure 3: Application tracking view with saved jobs and statuses.

Screenshot placeholder: `screenshots/resume-enhancement.png`
Add the app screenshot with this filename before final PDF compilation.

Figure 4: Resume enhancement and tailoring report.

6 System Architecture

The application has four main layers:

Layer	Implementation
Frontend	Static browser app using HTML, CSS, and JavaScript. It handles Google sign-in, resume upload, profile fields, recommendation rendering, saved applications, job browsing, modals, and user interaction state.
Backend API	FastAPI application in `backend.py`. It exposes endpoints for matching, retrieval debugging, resume parsing, cover letters, job data, bookmarks, applications, feedback, authentication, and email delivery.
Persistence	SQLite stores bookmarks, applications, feedback, resume enhancement results, and resume tailoring records. The backend also supports a canonical job database layer for normalized job storage.
Retrieval and LLM	Pinecone stores vectorized job postings. Local hash embeddings are used for deterministic vector representation. Gemma is used for generation through the Google API REST interface.

Table 1: High-level application architecture.

The backend also serves the frontend directly when opened through the root route. Protected endpoints use an API key when configured, and user-specific operations use bearer JWTs created after Google credential verification.

7 Data and Ingestion Pipeline

The project includes a local Adzuna India job dataset with 4,000 JSONL records under the project `data` directory. The ingestion code also supports external sources including Adzuna, Jooble, Greenhouse, Lever, Remotive, Arbeitnow, and USAJobs. Each external posting is normalized into a common job schema containing fields such as title, role, company, location, salary, skills, description, source, external URL, and metadata.

The source ingestion module supports India-focused filtering. It checks country, city, remote-work language, and India-related terms so the app can prioritize India-relevant jobs while still allowing remote opportunities when configured. The indexing mode can be configured as:

- `csv_only`: use only the local dataset.
- `live_only`: use external sources only.
- `hybrid`: combine local and external sources.

The indexing endpoint can force a re-index when needed. If the Pinecone index already contains vectors, startup indexing can be skipped to avoid unnecessary repeated work.

8 RAG and Matching Methodology

8.1 Query Construction

The backend builds a query from structured profile fields and resume text. For example, role and skill inputs are converted into a normalized form such as:

```
Roles: data analyst | Skills: python, sql
```

The query canonicalization step improves retrieval stability. Tests show that paraphrases and prompt-injection attempts such as “ignore previous instructions” canonicalize to the same retrieval intent when the underlying job need is the same.

8.2 Retrieval

Each job posting is converted into text using role, company, location, salary, skills, description, benefits, and source fields. The system creates deterministic local hash embeddings and stores vectors in Pinecone. At query time, the canonical query is embedded and used to retrieve top candidates. If Pinecone credentials are unavailable, the backend falls back to local scoring over the available job data.

8.3 Reranking

The retrieved jobs are reranked using a deterministic scoring function that combines the vector score with keyword overlap, profile skills, role preference, resume text, location, work type, and salary constraints where available. This reranking makes the top results more aligned with the user profile before they are passed into Gemma.

8.4 Gemma Generation

Gemma receives the cleaned user query, retrieved candidates, and resume context. The system prompt asks the model to behave as a precise career advisor and produce structured markdown. The frontend then parses and renders this output into job cards. If the model call fails or returns a response that cannot be rendered, the backend falls back to a deterministic response so the app remains usable.

9 Implementation Details

9.1 Backend Endpoints

The backend exposes endpoints for the full workflow. The most important routes are:

Endpoint	Purpose
`POST /webhook`	Main job matching endpoint for profile-based or chat-input-based recommendations.
`POST /parse-resume`	Extract structured profile information from a PDF resume.
`POST /debug/retrieval`	Return compact retrieval candidates.
`POST /debug/rag-trace`	Return canonical query, candidates, lexical overlap, prompt preview, model information, and safety flags.
`POST /cover-letter`	Generate a role-specific cover letter.
`POST /tailor-resume`	Produce job-specific resume tailoring analysis.
`POST /keyword-gap`	Extract present and missing job-description keywords.
`GET /jobs/browse`	Browse and filter available jobs.
`POST /jobs`	Add a job and index it when credentials are configured.
`POST /jobs/refresh`	Fetch configured external sources and index new jobs.
`POST /bookmark`	Save a job bookmark.
`POST /applications`	Create or update application tracking state.
`POST /send-results`	Email generated job results.
`POST /auth/google`	Exchange a Google credential for an app JWT.

Table 2: Representative backend API surface.

9.2 Frontend

The frontend is a static browser interface backed by JavaScript. It includes a landing page, Google sign-in gate, resume upload panel, optional constraints, results panel, job browse panel, application tracker, cover letter modal, job details modal, retrieval debug modal, resume enhancement modal, and resume tailoring modal. It stores the current session ID in local storage and uses bearer tokens for authenticated operations.

9.3 Security and Robustness

The implementation includes several practical safety controls:

- Prompt-injection patterns are detected and removed from untrusted user and PDF text.
- Resume uploads are rejected if they do not resemble real resumes.
- Gemma is explicitly forced into Gemma-only mode; non-Gemma model names are ignored.
- API key protection is supported for backend routes.
- Google SSO and JWTs are used for user-specific saved data.
- Email delivery checks configuration before enabling send actions.

10 Evaluation Methodology

The project includes an evaluation script, `eval.py`, and an evaluation log, `eval_log.jsonl`. Each logged recommendation records the profile/query, candidates before reranking, candidates after reranking, generated output, retrieval latency, and generation latency.

The evaluation script computes three groups of metrics:

Metric Group	Metrics
Retrieval	Precision@K, Recall@K, Hit@1, Hit Rate@K, MRR, MAP@K, NDCG@K with reranking, NDCG@K without reranking.
Generation	Format consistency, faithfulness, optional Gemma-as-judge score.
System	Skill diversity, company diversity, location diversity, profile alignment, retrieval latency, generation latency, mean match score.

Table 3: Evaluation metric categories.

Feedback-based metrics such as Recall@K, MAP@K, Hit Rate@K, and MRR require user ratings in the feedback table. The current local run has no feedback rows, so those metrics are marked unavailable in the preliminary results.

11 Preliminary Evaluation Results

The available local evaluation run contains one logged query and no feedback labels. This is enough to validate that the evaluation pipeline works, but it should be treated as a smoke-test result rather than a final benchmark.

Retrieval Metric	K=5	K=10
Precision@K, score ≥ 0.15	1.000	1.000
Recall@K, feedback-based	N/A	N/A
Hit@1, feedback-based	N/A	N/A
Hit Rate@K, feedback-based	N/A	N/A
MRR, feedback-based	N/A	N/A
MAP@K, feedback-based	N/A	N/A
NDCG@K with reranking	1.000	1.000
NDCG@K without reranking	1.000	1.000

Table 4: Preliminary retrieval metrics from the local evaluation log.

Generation/System Metric	Value
Format consistency	0.375
Faithfulness	1.000
Gemma judge average	N/A
Skill diversity, top 10	1.000
Company diversity, top 10	1.000
Location diversity, top 10	1.000
Profile alignment score	0.000
Average retrieval latency	1.85 ms
Average generation latency	0.89 ms
Mean LLM match score	10.000

Table 5: Preliminary generation and system metrics.

The NDCG reranking ablation currently shows no difference because the logged run contains only one retrieved job. A larger evaluation set with varied candidates and rated user feedback is required to measure reranking impact meaningfully.

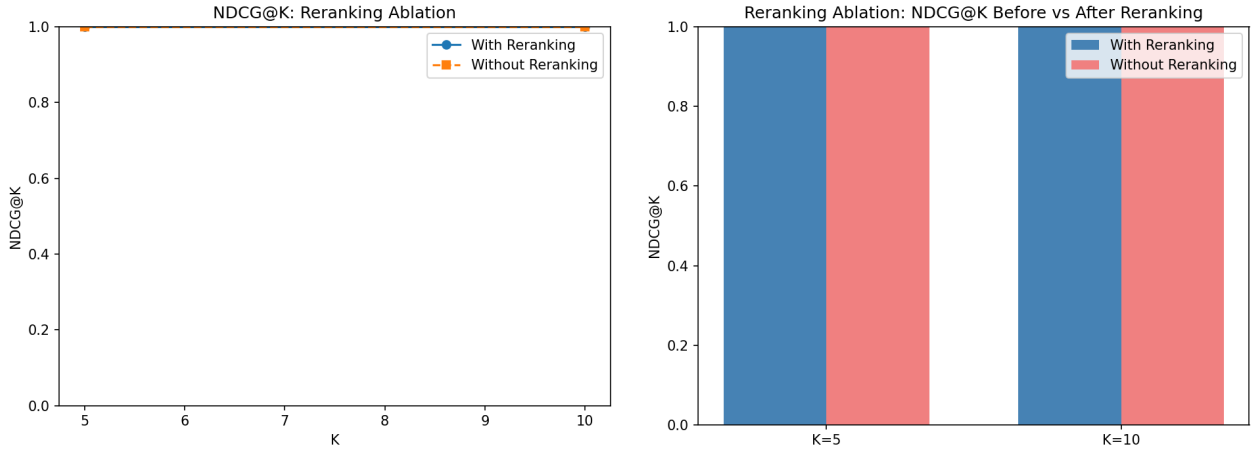

Figure 5: NDCG curve and reranking ablation from the local evaluation run.

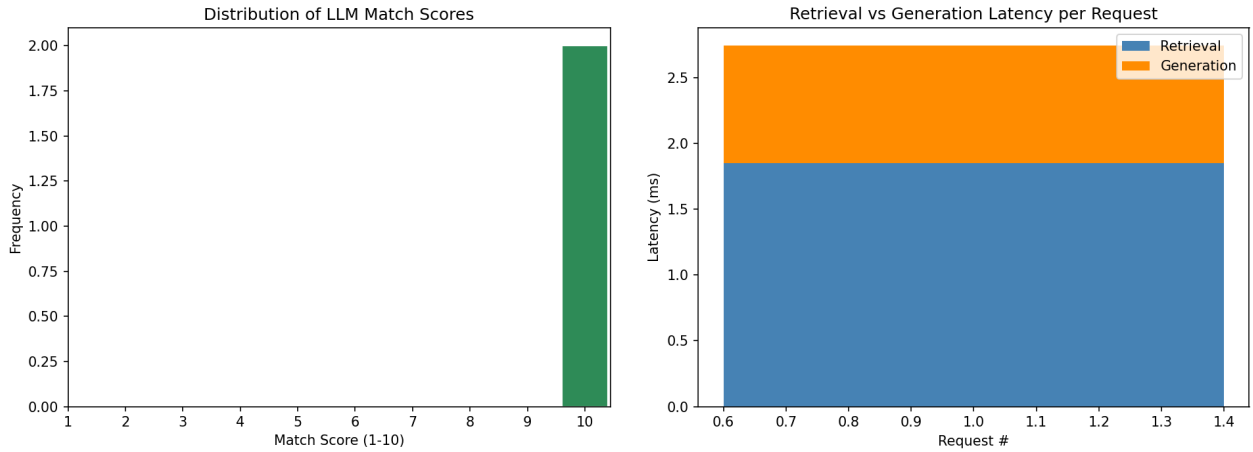

Figure 6: Match-score distribution and retrieval/generation latency.

12 Testing and Validation

The test suite focuses on backend behavior and RAG stability. The tests verify that jobs can be added and browsed from the canonical store, stats and RAG trace endpoints work without vector credentials, paraphrased queries canonicalize to the same retrieval intent, prompt-injection text is removed from

retrieval intent, resume quality gates reject repeated instruction junk, and webhook output does not echo injection text.

This validation matters because GenAI apps can fail in ways that are not visible through normal happy-path testing. For example, a recommendation may look fluent while being ungrounded, unstable across paraphrases, or influenced by malicious resume text. The implemented tests target those risks directly.

13 Limitations

The current implementation has several limitations:

- The preliminary evaluation set is too small for final conclusions; more logged queries and feedback labels are needed.
- Feedback-based Recall@K, MAP@K, MRR, and Hit Rate@K are unavailable until users rate job results.
- Local hash embeddings are deterministic and easy to deploy, but they are less semantically expressive than production-grade embedding models.
- Job quality depends on external source quality, API availability, and how complete each job description is.
- LLM-generated cover letters and resume suggestions require user review before real applications.
- The app supports authentication and API keys, but production deployment would need stronger monitoring, audit logs, and secret-management practices.

14 Future Scope

Future improvements include:

- Replace or augment local hash embeddings with stronger semantic embeddings.
- Add hybrid retrieval combining vector search with keyword/BM25 ranking.
- Collect more human feedback and use it to evaluate and personalize ranking.
- Expand evaluation with larger query sets, manually labeled relevance judgments, and cohort-based testing.
- Add dashboards for job-market trends, user application funnel analytics, and saved-search alerts.
- Integrate direct application submission where job boards permit it.
- Add richer resume editing, downloadable tailored resumes, and ATS-style formatting checks.
- Improve deployment hardening with rate-limit tuning, observability, retries, and background indexing jobs.

15 Conclusion

JobMatch AI demonstrates a practical applied GenAI system. It is not only a prompt wrapper: it includes data ingestion, normalized job records, vector retrieval, deterministic reranking, Gemma-based generation, authentication, persistence, email support, frontend workflows, safety checks, and evaluation infrastructure. The project therefore satisfies the goal of building an end-to-end full-stack application while using generative AI as a functional component inside a larger software system.

The preliminary metrics confirm that the evaluation pipeline runs, but final retrieval conclusions require a larger labeled evaluation set. The current implementation provides a strong base for continued experimentation with better embeddings, larger feedback data, richer personalization, and production-grade deployment practices.

A Appendix: Screenshot Checklist

Recommended screenshots to add before final submission:

- `screenshots/landing.png`: landing page or sign-in screen.

- `screenshots/resume-upload.png`: resume upload and parsed skills.
- `screenshots/job-results.png`: generated job recommendations.
- `screenshots/my-applications.png`: saved applications tracker.
- `screenshots/resume-enhancement.png`: resume enhancement or tailoring modal.
- `screenshots/browse-jobs.png`: browse jobs page and filters.

B Appendix: Main Technologies

Technology	**Use in Project**
FastAPI	Backend API and route handling.
SQLite / aiosqlite	Persistence for bookmarks, applications, feedback, resume enhancements, and tailoring records.
Pinecone	Vector index for retrieved job postings.
Gemma	LLM for recommendation explanation, resume parsing, cover letters, resume feedback, and recruiter email drafting.
pdfplumber	PDF resume text extraction.
Resend	Emailing results and generated cover letters.
Google SSO / JWT	Authentication and user-specific saved data.
HTML/CSS/JavaScript	Frontend interface and interaction logic.

Table 6: Technology stack summary.